Einführung in die Programmierung

Dozent: Prof. Dr. Michael Eichberg
Kontakt: michael.eichberg@dhbw.de , Raum 149B

Version: 1.3.0.1

Kontrollfragen: kontrollfragen.de.md.html
Folien: https://delors.github.io/prog-einfuehrung/folien.de.md.html
https://delors.github.io/prog-einfuehrung/folien.de.md.html.pdf
Fehler melden: https://github.com/Delors/delors.github.io/issues
KI Verwendung: Bei der Erstellung der Unterlagen wurden KI Assistenten (insbesondere Claude aber ggf. auch ChatGPT, Ollama mit Gemma/LLama/Qwen oder OpenCode mit Kimi/Qwen/Deepseek...) unterstützend eingesetzt. Dies erfolgte insbesondere zur Unterstützung bei der Generierung von Grafiken (d. h. SVG Dateien), oder um sich Übersichtstabellen generieren zu lassen. Weiterhin wurde KI zur allgemeinen Qualitätssicherung eingesetzt. Inhalte, die ggf. von der KI vorgeschlagen wurden, wurden im Falle der Übernahme explizit validiert und angepasst.

1. Einführung

Ziele

- Einfache Programme (in Java) schreiben zu können
- Grundlegende Werkzeuge für die Softwareentwicklung kennen lernen
- Die Fähigkeit zum *Computational Thinking* (d. h. zum algorithmischen Denken) zu erwerben
- Die Fähigkeit zu erwerben komplexe Probleme mittels des Prinzips *divide et impera* (teile und herrsche) zu lösen
- Ein paar grundlegende Begriffe der Softwareentwicklung kennen lernen

2. KI

KI Assistenten in der Softwareentwicklung

> **AI accelerates software development to breakneck speeds, but measuring that is tricky**
>
> [...]
>
> AI-assisted development is now the norm - 78% of survey respondents currently use AI in software development or plan to in the next two years, up from 64% in 2023.
>
> [...]
>
> Bringing in AI may be accelerating software development toward blinding speeds. Stunningly, most executives (69%) indicate they are shipping software twice as fast as last year.
>
> [...]
>
> —Juli 2024 - ZDNET

Auswirkungen von generativer KI

KI: Kannibalisieren sich die Programmierer?

Viele Branchen und Berufe wurden in den letzten Jahrzehnten durch IT umgekrempelt. Laut McKinsey und IDC trifft es wegen generativer KI jetzt die IT-Branche.

Laut McKinsey sollen in den kommenden drei Jahren bis zu 250 Milliarden US-Dollar in generative KI (GenAI) [...] investiert werden. Das betrifft vor allem die Codeerstellung, den IT-Helpdesk und die Testautomatisierung. [...]
Damit [GenAI] ließe sich die Produktivität der Softwareentwickler um bis zu 50 Prozent steigern. Das betrifft vor allem die Dokumentation, die Anpassung an neue Anforderungen und Refactoring. Das wird dazu führen, dass mehr Eigenentwicklung betrieben wird und weniger Standardsoftware zum Einsatz kommt. Dieser Trend werde dadurch verstärkt, dass Citizen-Developer demnächst nicht mehr Low-Code/No-Code nutzen, sondern nur noch in natürlicher Sprache und GenAI programmieren. [...]

—August 2024, Heise

Definition: Citizen Developer

Ein Citizen Developer ist kein professioneller Programmierer im engeren Sinn. D. h., niemand der Code in einer echten Programmiersprache erstellt. Es ist eine Person, die technisches Verständnis besitzt, um spezifische Anwendungen – insbesondere Geschäftsanwendungen – in einem wohldefinierten Umfeld selbst zu erstellen. Dazu setzt er *mächtige* Werkzeuge ein, die ihm die Arbeit der Codeerstellung abnehmen bzw. diese verstecken.

Der wirkliche Effekt von AI Coding Assistenten

Devs gaining little (if anything) from AI coding assistants

Code analysis firm sees no major benefits from AI dev tool when measuring key programming metrics, though others report incremental gains from coding copilots with emphasis on code review.

Many developers say AI coding assistants make them more productive, but a recent study set forth to measure their output and found no significant gains. Use of GitHub Copilot also introduced 41% more bugs, according to the study from Uplevel, a company providing insights from coding and collaboration data.

—September 2024, cio.com

METR previously published a paper which found the use of AI tools caused a 20% slowdown in completing tasks among experienced open-source developers, using data from February to June 2025. [...]

Unfortunately, given participant feedback and surveys, we believe that the data from our new experiment gives us an unreliable signal of the current productivity effect of AI tools. The primary reason is that we have observed **a significant increase in developers choosing not to participate in the study because they do not wish to work without AI**, which likely biases downwards our estimate of AI-assisted speedup.

AI Software Development – What Does The Data Say?

- Industry studies concur on the same trend, observed across hundreds of thousands of code bases and teams over multiple years – undoubtedly AI use increases coding activity very significantly, but that isn't translating into more value being created. By the time we look at not just what gets shipped, but what gets used, we see no discernible gains.
- The effective horizon of agent autonomy decays geometrically with reliability. The more reliable we need the generated code to be, the shorter the horizon. Even with quality gates capable of catching 99% of errors before they propagate and compound, for levels of reliability many of us would consider to be acceptable for release, horizons are extremely short.
- [...]
- A growing body of research finds a clear correlation between how easy code is for humans to understand and change reliably, and how easy it is for LLMs to do the same. Factors like readability, complexity and duplication have been shown to impact the accuracy of LLM predictions in a similar way to human programmers.
- [...]
- The energy and compute needed to train an LLM to be an order of magnitude more reliable – e.g., wrong 3% of the time instead of 30% – is 10^20 times what the current frontier models require. Don't expect significantly more reliable models any time soon. Any future gains in reliability will have to made by better context engineering (deciding what to include in the input) and more effective quality gates deciding what to do with the output- and that's exactly what we're seeing AI companies focusing on these days. Models may get more powerful, but not significantly more reliable.
- [...]

—August 2026, Jason Gorman

AI Assistenten und Grundlagen der Programmierung

- Verwenden Sie keine AI Assistenten, um sich die Aufgaben lösen zu lassen.

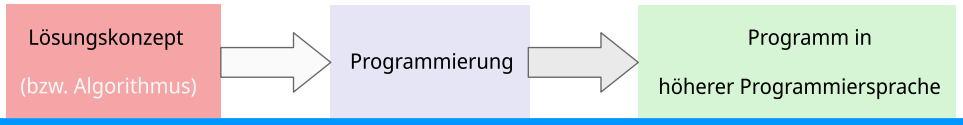
  Sie lernen dadurch effektiv nichts!
- Die Aufgaben werden immer nur Dinge verlangen, die gelehrt wurden.
- In der Klausur/Prüfung steht Ihnen auch kein AI Assistent zur Verfügung.

> ▲ Achtung!
>
> Später müssen **Sie** in der Lage sein, den Code, der von Assistenten generiert wurde, zu verstehen und zu validieren.
>
> Ohne ein tiefgreifendes Verständnis ist dies nicht möglich. Sie sind bwz. werden verantwortlich sein!

3. Programmieren?

Was ist Programmieren?

Programmieren bezeichnet das Formulieren eines Lösungskonzeptes (Algorithmus) in einer Programmiersprache.



Hinweis

Ohne Programmierkenntnisse ist es unmöglich zu beurteilen, wie komplex eine Aufgabenstellung ist.

Grundlagen der Programmierung[1]

01 Eine präzise Spezifikation von dem was wir wollen/benötigen.

02 Die Verifikation, dass wir bekommen haben, was wir wollen.

03 Die Fähigkeit in kleinen Schritten Fortschritt zu machen.

---

[1] Dave Farley, Modern Software Engineering, 9. Oktober 2025

Programmiersprachen

Tiobe Index für September 2025

Programmiersprache	Anteil
Python	25.98%
C++	8.8%
C	8.65%
Java	8.35%
C#	6.38%
JavaScript	3.22%
Visual Basic	2.84%
...	...

Pypl Index für September 2025

Programmiersprache	Anteil
Python	29.6%
Java	14.72%
C/C++	9.27%
JavaScript	6.79%
R	5.26%
C#	4.81%
Objective-C	4.17%
...	...

Programmiersprachen und -paradigmen

- Maschinensprachen (Zahlenkombinationen als Befehle)

  (Eine Programmierung in Maschinensprache wird heute nicht mehr durchgeführt.)
- Assemblersprachen (wie Maschinensprachen nur besser lesbar)

  (Es gibt auch heute noch (vereinzelt) Anwendungsbereiche für Programmierung in Assembler.)
- Höhere Programmiersprachen:
  - Prozedurale Programmiersprachen (z. B. C - noch immer sehr verbreitet)
  - Objekt-orientierte Programmiersprachen (z. B. C++, Java, ... - sehr verbreitet)
  - Funktionale Programmiersprachen (z. B. Clojure, F#, ...) - in ausgewählten Anwendungsgebieten
  - Logische Programmiersprachen (z. B. Prolog - nur noch in (sehr) speziellen Anwendungsgebieten)
  - Sprachen, die verschiedenste Konzepte unterstützen (z. B. Java, Scala, Kotlin, Python, TypeScript, ... - der aktuelle Standard)
- Sprachen für spezielle Anwendungsgebiete (z. B. SQL oder Low-code Ansätze)

  (Meist deklarative Sprachen, die für spezielle Anwendungsgebiete entwickelt wurden.)

Bei deklarativen Sprachen beschreibt man nicht wie etwas berechnet wird, sondern was berechnet werden soll. Maschinensprachen und Assemblersprachen sind imperative Sprachen. Höhere Programmiersprachen sind meist imperative Sprachen, aber es gibt auch deklarative Sprachen (insbesondere Prolog bzw. Datalog).

Seit ca. 10 bis 15 Jahren kann man beobachten, dass die Grenzen zwischen den verschiedenen Paradigmen immer mehr verschwimmen und einst rein objekt-orientierte Sprachen auch funktionale Elemente enthalten und umgekehrt.

Sprachen für spezielle Anwendungsgebiete werden auch *Domain-specific Languages* (DSLs) genannt.

Was ist ein Algorithmus?

Definition: Algorithmus

Eine exakte, endliche Vorschrift zur schrittweisen Lösung eines (lösbaren) Problems.

Die Beschreibung eines Algorithmus erfolgt mit Hilfe eines wohl definierten Formalismus und ggf. mit Hilfe weiterer (elementarer) Algorithmen.

Es ist zum Beispiel nicht möglich alle reellen Zahlen aufzuzählen - das Problem ist nicht lösbar und es kann kein Algorithmus angegeben werden!

Klassifizierung von Algorithmen

Entscheidungsprobleme:

Für eine Eingabe eine bestimmte Eigenschaft feststellen.

Beispiel: Ist eine gegebene Zahl eine Primzahl?

Aufzählungsprobleme:

Alle Daten mit einer gegebenen Eigenschaft aufzuzählen.

Beispiel: Zähle alle Primzahlen kleiner als 100 auf!

Berechnungsprobleme:

Eine gegebene Eingabe in eine Ausgabe zu transformieren.

Beispiel: Addition zweier gegebener Zahlen

Beispiel: Berechnung der Fakultät (rekursiv)

Mathematisch

Input: natürliche Zahl (inkl. 0)

Output: natürliche Zahl

$$\text{fak}(n) = \begin{cases} 1 & \text{if } n = 0 \\ n \cdot \text{fak}(n-1) & \text{if } n > 0 \end{cases}$$

Java Programm bzw. Skript

```java
long fak(long n){
  if (n == 0) return 1;
  else return n * fak(n-1);
}
```

Zentrale Eigenschaften von Algorithmen

Allgemeinheit: Ein Algorithmus löst eine Klasse von Problemen. Die konkrete Instanz wird über die Eingabeparameter/Parametrisierung festgelegt.

Endlichkeit: Die Beschreibung des Algorithmus bzgl. der Verarbeitungsschritte und (Eingabe-)Daten ist endlich und ermöglicht eine Ausführung mit endlichen Ressourcen.

Determiniertheit: Der Algorithmus führt bei gleichen Eingaben immer zu gleichen Ausgaben.

Ausführbarkeit: Der Algorithmus besteht aus einer Folge von elementaren (ausführbaren) Schritte. Diese werden vom Prozessor ausgeführt. Elementare Operationen sind z. B.:

- einfache arithmetische Operationen wie Addition, Subtraktion, Division etc.
- Vergleiche
- Zuweisungen
- etc.

Terminiertheit: Das Ergebnis liegt nach endlich vielen Schritten vor.
(Dies bedeutet aber nicht, dass das Problem auch praktisch lösbar ist.)

Komplexität: Zeit und Platzbedarf sind endlich und in einem gewissen Rahmen abschätzbar.
Normalerweise versucht man den besten, schlechtesten und durchschnittlichen Fall zu bestimmen in Abhängigkeit von der Eingabegröße. (Insbesondere Thema des nächsten Semesters).

Diese Angaben erfolgen unabhängig von einer konkreten Implementierung bzw. Verwendung einer bestimmten Programmiersprache oder Hardware.

Determinismus: Jeder (Teil-)schritt führt bei gleichen Eingaben immer zu gleichen Ausgaben.

Nicht jeder Algorithmus, der die Eigenschaft der **Determiniertheit** erfüllt, ist auch deterministisch. Bei einem deterministischen Algorithmus führt jeder (Teil-)schritt bei gleichen Eingaben immer zu gleichen Ausgaben, aber dies muss (in bestimmten Fällen) nicht immer erfüllt sein und der Algorithmus kann dennoch determiniert sein.

Insbesondere im Bereich der Kryptographie basieren viele Algorithmen darauf, dass die Ver-/Entschlüsselung nur dann effizient durchführbar ist, wenn man den Schlüssel kennt. Ist der Schlüssel nicht bekannt, dann kann immer noch ein **terminierender Algorithmus** angegeben werden, der verschlüsselte Daten entschlüsselt, aber dieser ist nicht effizient in sinnvoller Zeit durchführbar.

Die **Komplexität eines Algorithmus** bestimmt ganz maßgeblich wofür dieser Algorithmus eingesetzt werden kann. Wir können zum Beispiel Algorithmen wie folgt unterscheiden:

- konstante Komplexität (d. h. der Algorithmus benötigt unabhängig von der Größe der Eingabe immer gleich lange.)
- logarithmische Komplexität
- lineare Komplexität
- quadratische Komplexität
- exponentielle Komplexität (d. h. praktisch nicht anwendbar; häufig sucht man nach alternativen Algorithmen, die auf Heuristiken basieren. Zum Beispiel für das Erfüllbarkeitsproblem (🇺🇸 *Satisfiability*) in der Aussagenlogik.)

Beispiel: Berechnung bzw. Approximation von e

Mathematisch (exakt)

Output: reelle Zahl

$$e = \sum_{i=0}^{\infty} \frac{1}{i!} = \frac{1}{0!} + \frac{1}{1!} + \frac{1}{2!} + \dots$$

Java Programm bzw. Skript

```java
BigDecimal e(int steps) {
  BigDecimal e = BigDecimal.ZERO;
  while (steps >= 0) {
    e = e.add(
        BigDecimal.ONE.divide(
          new BigDecimal(fak(steps)
        ),
        MathContext.DECIMAL128));
    steps--;
  }
  return e;
}
```

---

e steht hier für die eulersche Zahl.

In diesem Fall wurde folgende Implementierung der Fakultät verwendet:

```java
import java.math.BigInteger;

BigInteger fak(int n) {
  if (n == 0)
    return BigInteger.valueOf(1);
  else {
    var bn = BigInteger.valueOf(n);
    return fak(n-1).multiply(bn);
  }
}
```

✎ Übung: Primzahltest

- Schreiben Sie in natürlicher Sprache einen Algorithmus, der eine beliebige natürliche Zahl testet ob diese eine Primzahl ist.

  Achten Sie darauf, dass der Algorithmus die vorher diskutierten Eigenschaften erfüllt.

- Beschreiben Sie die Komplexität Ihres Algorithmus.

  D. h., wie viele Ressourcen (z. B. Rechenschritte) sind in Abhängigkeit von der Eingabe notwendig?

(Programmier-)sprachen - Unterteilung

natürliche Sprachen:

- Dienen der Kommunikation zwischen Menschen und sind häufig mehrdeutig.
- In vielen Fällen ist die Bedeutung eines Satzes abhängig vom Kontext.

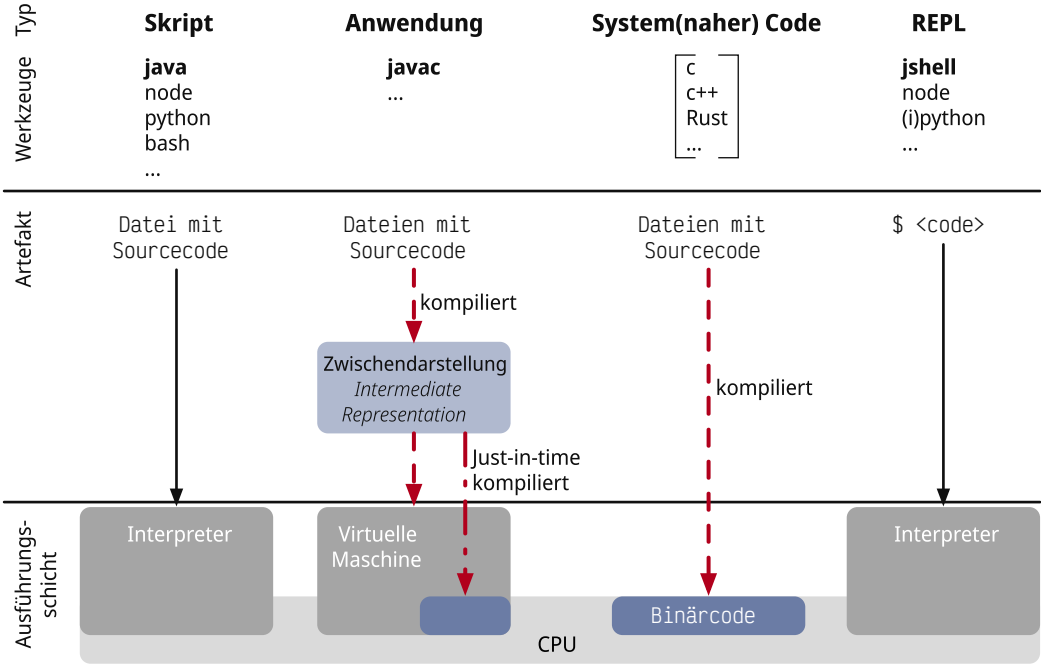
formale Sprachen:

- Dienen der eindeutigen Beschreibung von Sachverhalten; sind präzise und eindeutig.
- Können ggf. automatisch ausgewertet werden.
- Programmiersprachen sind formale Sprachen zur Beschreibung von Algorithmen.

Einsatzbereiche verschiedener Programmiersprachen

- Systemprogrammierung
- Anwendungsprogrammierung / Web-Entwicklung
- Skriptsprachen

Arten der Programmierung



Syntax und Semantik von formalen Sprachen

Syntax: definiert welche Sätze in der Sprache gültig sind. Die Syntax wird durch eine Grammatik formal und präzise beschrieben.

Semantik: definiert die Bedeutung der Sätze; wenn dies möglich ist. Nicht jeder syntaktisch korrekte Satz hat eine Bedeutung.

◆ Bemerkung

Häufig wird die Semantik „nur“ in einem Standard oder „sogar nur“ durch eine Implementierung festgelegt.

Formale Sprachen: Beispiel in einer *EBNF* Variante

Syntaktisch gültiger Satz:

Sie geht nach Hause.

Syntaktisch gültig, aber semantisch falscher Satz:

Tim schwimmt auf den Mond.

Syntaktisch ungültige Sätze:

Sie fährt nach Hause in die Schule.

Tim geht in die Schule

```
Satz      = Subjekt WS Prädikat WS Objekt "."
Subjekt   = "Tim" | "Sie"
Prädikat  = "geht" | "fährt" | "schwimmt" | "fliegt"
Objekt    = "nach Hause" | "in die Schule" | "auf den Mond"
WS        = " "
```

Es gibt zahlreiche Varianten der EBNF (Extended Backus-Naur Form). Die grundlegenden Ideen und Konzepte sind jedoch überall gleich.

Beispiele für verschiedene Fehler in Java Programmen

```
int fak(long n){
    if (n == 0)
        return 1l // ';' expected
                  // ⇒ "Syntaktischer Fehler"
    else
        return n * fak(n-1);
                // incompatible types: possible lossy conversion from long to int
                // ⇒ "Semantischer Fehler"
}
```

Extended-Backus-Naur-Form (EBNF)

Die EBNF dient der Beschreibung kontext-freier Grammatiken.

Verwendung	Notation	Bedeutung
Definition	=	
Konkatenation	,	
Terminierung	;	Ende der Def.
Alternative	|	
Optional	[...]	0 oder 1mal
Wiederholung	{ ... }	0 oder mehrfach
Gruppierung	(...)	
Kommentar	(* ... *)	
Terminalsymbol	"Terminal"	

Beispiel

```
Ausdruck =
    Ziffer,
    {   ("+" | "-"),
        Ziffer };
Ziffer =
    "1" | "2" | "3" |
    "4" | "5" | "6" |
    "7" | "8" | "9" |
    "0";
```

Die Beschreibung einer Programmiersprache in EBNF besteht aus einer Startregel und einer Menge von weiteren Regeln sowie Terminalen, die die Syntax der Sprache beschreiben. Die Terminalen sind die Basiswörter der Sprache („reservierte Wörter“).

Bei kontext-freien Sprachen kommt auf der linken Seite einer Regel genau ein Nichtterminal vor, auf der rechten Seite können beliebig viele Nichtterminale und Terminale vorkommen.

Was genau „kontext-frei“ bedeutet, ist Gegenstand fortgeschrittener Vorlesungen. Für den Moment reicht es zu wissen, dass dies gewisse Einschränkungen für die Regeln bedeutet. Diese Einschränkungen aber das Verarbeiten zumindest stark vereinfachen und dadurch eine effiziente und eindeutige Verarbeitung ermöglichen.

✎ Übung: EBNF verwenden

3.1. Binärzahlen in Java

Folgend wird die Syntax von Binärzahlen (`BinaryNumeral`) in Java beschrieben.[2]

```
BinaryNumeral = "0", ("b" | "B"), BinaryDigits
BinaryDigits = BinaryDigit | (BinaryDigit [BinaryDigitsAndUnderscores] BinaryDigit)
BinaryDigit = "0" | "1"
BinaryDigitsAndUnderscores = BinaryDigitOrUnderscore {BinaryDigitOrUnderscore}
BinaryDigitOrUnderscore = BinaryDigit | "_"
```

Welche der folgenden Zahlen sind gültige Binärzahlen in Java? Falls nicht, warum?

`0b1010`	`0B1010_0`
`0b1_0_1_0`	`0B1010_0_`
`0B_1010`	`0b1010_0_1`
`0b1010_`	`0_b101`

---

[2] Die von der JLS (Java Language Specification) verwendete Syntax wurde hier adaptiert an die zuvor gezeigte Variante. Weitere Änderungen wurden nicht vorgenommen.

✎ Übung: EBNF erweitern

3.2. EBNF für einfache Ausdrücke

Erweitern Sie die EBNF für mathematische Ausdrücke, um die Möglichkeit Zahlen beliebiger Länge anzugeben und auch Ausdrücke (mathematisch korrekt) zu klammern. D. h. Ihre erweiterte Grammatik soll folgende Ausdrücke zulassen:

```
12+25
```

```
13-4-(4+5)
```

Bonus

Erweitern Sie die EBNF so, dass auch einfache Fließkommazahlen erlaubt sind (z. B. „`1, 2`“ oder „`0, 999`“). Achten Sie darauf, dass keine ungültigen Zahlen wie „`1, `“ oder „`1, 2, 3`“ erlaubt sind.

Voraussetzungen

Zu installieren (für den Anfang):

- Java 27 JDK (Java Development Kit)

  https://adoptium.net/en-GB/temurin/releases/?version=27

  oder

  https://www.azul.com/downloads/?package=jdk#zulu

  (Java 24 oder älter ist nicht ausreichend.)
- Visual Studio Code, Eclipse Theia, IntelliJ IDEA, ZED oder Eclipse

> ▲ Achtung!
>
> Deaktivieren bzw. deinstallieren Sie ggf. jegliche KI Assistenten, damit Ihnen die Aufgaben nicht direkt gelöst werden! Alternativ können Sie einen einfachen Texteditor mit einfachem Syntax-highlighting benutzen. (Mac: CotEditor; Windows: Notepad++, Linux: Kate oder Gedit)